AeroFlux Stage 2: A Scalable, Portable Big-Data Platform for Real-Time Flight-Delay Intelligence

Formal Project Proposal

Jonathan Wilson Ryan Sollom

July 18, 2026

Course: AIT 614 Sec 001
Instructor: Dr. Duoduo Liao
Institution: George Mason University

1 Introduction

Flight delays are among the most costly and disruptive failures in the U.S. National Airspace System (NAS), and they rarely occur in isolation. A single late arrival propagates downstream through the same aircraft's rotation and through shared airport departure and arrival queues, so a local disturbance becomes a network-wide disruption.

Although airlines, airports, and air traffic authorities increasingly use real-time and predictive decision-support systems, these capabilities are often specialized around particular organizations, operational domains, or stages of flight. For example, the FAA's Traffic Flow Management System supports demand-and-capacity management across the National Airspace System, while Terminal Flight Data Manager integrates surface and national traffic-management data to improve airport operations (Federal Aviation Administration, 2020, 2024c). Commercial platforms such as Sabre Mosaic support airline disruption recovery, and Assaia ApronAI uses computer vision and machine learning to monitor aircraft turnarounds and predict departure readiness (Assaia International AG, n.d.; Sabre Corporation, n.d.). Opportunities therefore remain for integrated research platforms that fuse multiple live aviation data streams, model delay propagation across the broader flight network, and support forecasting, simulation, data science, and AI-assisted analysis.

Scope. The long-term vision for AeroFlux is ambitious and continues to evolve. Stage 2 therefore focuses on building a scalable, modular, configurable, portable, and decoupled big-data platform that can ingest, fuse, process, and analyze aviation and weather data in real time. This platform is intended to establish the technical foundation for future AeroFlux capabilities.

Flight-delay prediction serves as the primary use case, but the architecture is designed to support additional applications, including congestion analysis, delay-propagation modeling, operational monitoring, decision support, and agentic AI. Stage 1 established a historical machine-learning pipeline and interactive prototype (Wilson et al., 2026). Stage 2 extends that work by introducing a live data platform, comparing baseline and advanced predictive models, and developing an explainable analyst agent as an initial prototype.

Technical challenges. Building this platform requires solving several non-trivial problems:

(1) Ingesting heterogeneous, error-prone streaming sources (FAA SWIM XML, ADS-B, weather) without data loss;
(2) *Entity resolution* across systems that share no universal key—for example, the aircraft tail number is not published in SWIM flight data and must be recovered from ADS-B which itself has limitations due to lack of coverage;
(3) Unstable identifiers, since a flight's `flight_ref` changes on plan amendment while its Globally Unique Flight Identifier (GUFI) remains stable;
(4) Train-serve skew, because historical labels (Bureau of Transportation Statistics) and live features are measured differently; and
(5) Delivering all of this at demo quality on a portable, low-cost footprint.

Several of these challenges have already been partially addressed through a working end-to-end prototype pipeline. However, the proposed implementation is intended to reduce and manage

these limitations rather than eliminate every possible source of missing data, ambiguity, or operational uncertainty.

2 Related Work

Machine-learning delay prediction. Prior work frames delay prediction as supervised classification or regression over historical records. The review by Sternberg et al. (2017) describes the major approaches used for flight-delay prediction, while more recent work continues to evaluate machine-learning algorithms and class-imbalance handling (AlBassam & AlShahrani, 2025). These approaches can achieve strong predictive performance on curated historical datasets. However, when used in isolation, they may remain largely retrospective, rely on a limited set of data sources, and model flights as independent observations. These assumptions can restrict their ability to capture the temporal, spatial, and operational dependencies through which delays propagate across the aviation network.

Delay-propagation modeling. A separate literature models how delays ripple through airline schedules. Beatty et al. (1999) introduced an early method for evaluating delay propagation through an airline schedule, Wong & Tsai (2012) applied a survival model, and Kafle & Zou (2016) proposed an analytical-econometric approach. These methods capture important propagation mechanisms but are generally offline and disconnected from real-time streaming prediction. Belcastro et al. (2016) demonstrated scalable data mining for flight-delay prediction, establishing the value of big-data tooling while leaving room for tighter integration with real-time data fusion.

Operational data fusion. The state of the art in operational systems includes NASA's Airspace Technology Demonstration 2 (ATD-2) *Fuser*, which fuses multiple aviation data sources into a unified flight record (National Aeronautics and Space Administration, 2021). Its strength is production-grade fused operational data; its limitation for this project is that it is not designed as a lightweight, open, extensible ML/AI experimentation platform. This project actually leverages some of the high level concepts used in ATD-2 to create our own fuser.

Emerging aviation intelligence systems. Recent operational systems show that the field is moving beyond static delay prediction toward real-time prediction, simulation, and decision support. FlightAware Foresight represents production ML-driven predictive flight tracking, including arrival-time, taxi-out, and runway predictions (FlightAware, {n.d.}). NASA's NAS Digital Twin and Project Bluebird extend this direction toward replayable and probabilistic simulation environments for live or historical airspace operations and AI-agent evaluation (Lauderdale et al., 2024; Pepper et al., 2026). Commercial platforms such as Airspace Intelligence PRESCIENCE, Lufthansa Systems NetLine/Ops++ aiOCC, and Amadeus Flight Operations Control show the market moving toward 4D operating-domain twins, AI-assisted operations control, disruption recovery, and resource-aware decision support (Airspace Intelligence, {n.d.}; Amadeus, {n.d.}; Lufthansa Systems, 2025). Recent prototypes such as FlightSense further combine rotation-chain features, real-time MLOps, and conversational/agentic interfaces for delay prediction (Shelke et al., 2026), while the FAA AI safety roadmap highlights the need for assurance, incremental deployment, and

aviation-specific safety methods when adding AI to operational contexts (Federal Aviation Administration, 2024a). Together, these efforts motivate AeroFlux as a lightweight, SWIM-native experimentation platform that connects real-time data fusion, propagation-aware ML, replay, and explainable/agentic analysis rather than treating delay prediction as a standalone model.

Relevance of this approach. AeroFlux Stage 2 bridges these strands. It couples lightweight, Fuser-inspired real-time fusion and entity resolution with propagation-aware feature engineering and an explainable retrieval-augmented agent—integrating capabilities that prior work often treats separately.

3 Objectives

1. Ingest and fuse FAA SWIM, ADS-B, and weather data into a validated, per-flight canonical state in real time.
2. Deliver a portable, decoupled platform that runs unchanged on a laptop, school hardware, or any cloud VM (container-first, cloud-agnostic).
3. Compare a baseline model against an advanced, propagation- and weather-aware model for arrival-delay prediction.
4. Evaluate both **predictive** performance and **system** performance.
5. Develop and evaluate a starter Aviation Operations Analyst that uses retrieval, model invocation, explainability, and structured tool calling to produce evidence-grounded disruption-investigation briefs without making operational decisions.
6. Define stable tool interfaces that can later support knowledge graphs, operational-event replay, simulation, MCP integration, and specialized agent workflows without requiring replacement of the underlying data platform.

4 Datasets

The platform fuses live and historical aviation and weather sources, plus reference dimensions and a documentation corpus, summarized in Table 1. FAA SWIM provides the authoritative live NAS flight state (Federal Aviation Administration, 2024b); ADS-B supplies the physical airframe identity (ICAO 24-bit address and registration) needed to link an aircraft's successive legs, drawn from community feeds (adsb.lol, 2026; airplanes.live, 2026), whose research value is demonstrated by large-scale networks such as OpenSky (Schäfer et al., 2014); NOAA weather—a leading delay driver—is drawn from live METAR via the Aviation Weather Center (National Oceanic and Atmospheric Administration, 2026) for serving and the NCEI Integrated Surface Database for historical training (National Centers for Environmental Information, 2026); and the BTS On-Time Performance dataset, which includes actual gate times and tail numbers, provides historical training labels (Bureau of Transportation Statistics, 2026). Reference dimensions for airports and airlines (OpenFlights, 2026; OurAirports, 2026) support entity resolution, code normalization, and timezone alignment.

Table 1: AeroFlux Stage 2 data sources, retention policies, and estimated storage requirements. Estimated sizes are project-planning ranges rather than published source totals and depend on geographic scope, ingestion frequency, selected fields, compression, and duplicate handling.

Source	Type	Role	Retention	Estimated retained size
FAA SWIM (TFMS)	Streaming XML	Live flight state (primary)	48 h	Approximately 5–25 GB
ADS-B (airplanes.live / adsb.lol)	Streaming JSON	Airframe and tail resolution	48 h	Approximately 2–10 GB
NOAA weather — METAR (AWC) + NCEI ISD hourly	Polled JSON / text	Weather: METAR for serving, NCEI for training	48 h	Approximately 50–250 MB live
BTS On-Time Performance	Historical CSV / Parquet	Historical training labels	Persistent	Approximately 20–40 GB as CSV or 4–10 GB as Parquet
Aviation documents (FAA, SWIM dictionaries, and related references)	PDF / HTML / text	RAG documentation corpus	Persistent	Approximately 0.5–2 GB, including extracted text and embeddings
Reference dimensions (airports, airlines)	Static CSV	Entity resolution, ICAO/IATA + timezone normalization	Persistent	< 50 MB

A raw SWIM message (abridged) is shown in Listing 1; the corresponding fused, validated canonical record produced by the ingestion pipeline is shown in Listing 2. Each SWIM document explodes into one record per flight, which the pipeline fuses by GUFI across many messages over time.

5 Methodology and Proposed Solution

Architecture. AeroFlux Stage 2 is a streaming lakehouse: heterogeneous aviation and weather sources are ingested continuously, processed through a stream-processing core, and persisted across bronze → silver → gold tiers. Every component runs as a container, so the platform deploys unchanged on a laptop, university hardware, or a single cloud VM, with managed AWS services as an optional scale path rather than a requirement. The full

Listing 1 Raw SWIM TFMS flight message (abridged).

```
<fdm:fltdMessage acid="AAL2033" airline="AAL" arrArpt="KLGA" depArpt="KCLT"
   flightRef="150976233" msgType="FlightModify" ...>
  <nxcm:flightStatus>PLANNED</nxcm:flightStatus>
  <nxcm:aircraftModel>A320</nxcm:aircraftModel>
  <nxcm:flightTimeData airlineOutTime="2026-07-04T15:39:00Z"
                       airlineInTime="2026-07-04T17:35:00Z"/>
</fdm:fltdMessage>
```

Listing 2 Fused, schema-validated canonical flight-instance record.

```
{ "flight_instance_id": "KN6770564K", "callsign": "AAL2033",
  "flight_number": "AA2033", "carrier_name": "American Airlines",
  "origin": "KCLT", "destination": "KLGA", "aircraft_type": "A320",
  "scheduled_gate_departure": "2026-07-04T15:39:00Z",
  "estimated_arrival": "2026-07-04T17:41:00Z", "flight_status": "PLANNED",
  "resolution_status": "airline_resolved", "schema_version": "1.0" }
```

architecture is shown in Figure 1.

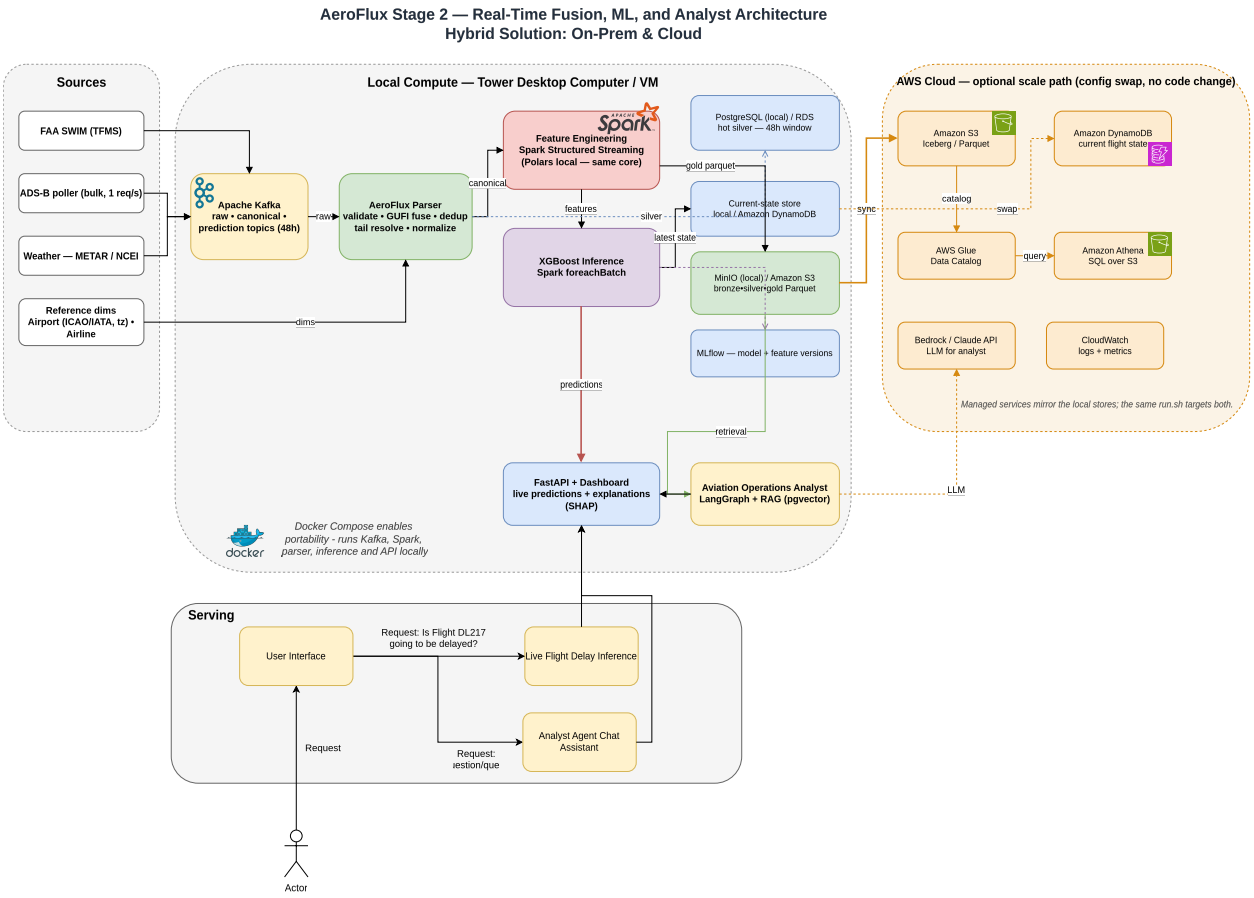

Figure 1: AeroFlux Stage 2 system architecture: Real-Time Fusion, ML, and Analyst Architecture. Hybrid Solution: On-Prem and Cloud.

Ingestion and streaming. Dedicated services publish each source—SWIM flight messages, ADS-B, and weather—to a dedicated Apache Kafka topic with 48-hour retention, decoupling producers from consumers and providing a replayable event history. A Kafka-consuming stream processor applies the `aeroflux_parser` library to validate each message against a strict schema contract and fuse messages into per-flight canonical records—keyed on the stable GUFI, with ADS-B used to resolve the aircraft tail that SWIM omits. The same feature and inference core runs on Polars for single-node deployment and executes within Apache Spark Structured Streaming's `foreachBatch` for distributed scale-out, so the platform spans laptop to cluster without a rewrite.

Storage tiers and retention. Processing follows a bronze → silver → gold progression. Raw and fused per-flight state are held hot in PostgreSQL on a rolling 48-hour window; current per-flight predictions are served from a NoSQL store (Amazon DynamoDB in the cloud); and canonical and gold feature/label tables are written as partitioned Parquet to a MinIO / Amazon S3 data lake, queryable in place through Amazon Athena over an AWS Glue catalog. Short retention keeps the streaming footprint small and low-cost, while the lake retains durable history for training and replay. Because storage backends sit behind configuration, the local stack (PostgreSQL, MinIO) swaps to managed cloud services (RDS,

S3) with no code change.

Machine learning. The pipeline establishes a logistic-regression *baseline* on flight-level features and an advanced gradient-boosted model (XGBoost) that improves on it by adding propagation, weather, and rolling airport-demand features (Beatty et al., 1999; Chen & Guestrin, 2016; Kafle & Zou, 2016). Feature engineering is expressed once in a source-agnostic core, so historical (BTS) training and live serving compute identical features by construction; that core runs on Polars locally for development and within Spark at scale. For historical training, weather is drawn from a cached NOAA NCEI archive; live serving uses METAR observations mapped into the same feature schema, preserving train/serve parity at the feature level. Both models target a 15-minute arrival-delay classification, are tracked in a run registry (with optional MLflow integration), and are explained with SHAP (Lundberg & Lee, 2017).

AI orchestration prototype. A lightweight *Aviation Operations Analyst* uses retrieval-augmented generation (Lewis et al., 2020) and a constrained LangGraph workflow over a curated aviation-document corpus to produce cited, evidence- grounded briefs; it does not touch live streaming data or make operational decisions, and is evaluated on a small set of reviewed questions for groundedness, citation accuracy, and appropriate tool use. As an optional stretch goal, a read-only Model Context Protocol (MCP) server may expose these tools through a standardized interface.

Evaluation. We evaluate at two levels, each against an explicit baseline.

Predictive (model) baseline. The logistic-regression classifier is the baseline; the advanced XGBoost model is the improvement. Both target a 15-minute arrival-delay classification and are compared on ROC-AUC, PR-AUC, F1, and Brier score with calibration, using point-in-time features to prevent leakage. We also report train/serve behavior honestly: because rotation and recent-delay features are dense in historical training but sparse in live serving—airframe resolution depends on ADS-B coverage—live predictions exhibit a distribution shift we quantify and discuss, with probability calibration identified as future work.

System baseline. The system is evaluated on sustained throughput (messages per second), ingestion-to-gold latency, airframe-resolution coverage (the share of live flights linked to an ADS-B airframe), fusion completeness (per-flight field fill rates), and resource footprint. Baseline measurements are taken at initial deployment and compared against achieved performance after optimization, demonstrating that the pipeline moves data correctly and at scale through every tier.

6 Development Environment

The stack (Table 2) is entirely open-source and container-first. Every component runs as a Docker Compose service, so the platform deploys unchanged on a laptop, university hardware, or a single cloud VM; managed AWS services (MSK, EMR, SageMaker) are an optional scale path rather than a requirement. MinIO provides an S3-compatible object store that swaps to

real S3 with no code change. Short retention keeps storage and cost minimal (target budget: a modest VM plus a few dollars of LLM API usage).

Table 2: Development environment and technology stack.

Category	Tools
Language	Python 3.11
Streaming	Apache Kafka, Spark Structured Streaming
Storage	MongoDB, PostgreSQL + pgvector, MinIO / Amazon S3
Historical replay	Kafka offsets and immutable Parquet event history in MinIO / S3
ML	scikit-learn, XGBoost, Spark ML, MLflow, SHAP
AI orchestration	LangGraph constrained workflow with read-only tool calling
LLM / RAG	Claude API or Amazon Bedrock; pgvector semantic retrieval
Agent tools	Document search, operational-data query, model inference, SHAP explanation, event reconstruction
Serving / UI	FastAPI, Dash / Plotly
Orchestration	Prefect, Airflow or other
Interoperability	Optional read-only AeroFlux MCP server
Packaging / Platform	Docker Compose; optional AWS EC2, S3, and EMR Serverless

7 Project Tasks and Timeline

The minimum implementation is scoped to approximately one month (Table 3), building on Stage 1's completed ingestion, fusion, entity-resolution, schema contract, and baseline model. Future capabilities—graph analytics, simulation, digital twin, and a multi-agent system—are explicitly out of scope for this timeline.

Table 3: One-month project timeline and effort allocation.

Week	Tasks	Est. hours
1	Complete streaming and weather integration; prepare MongoDB records, documentation corpus, and two or three historical investigation scenarios	20

Week	Tasks	Est. hours
2	Feature engineering; baseline and advanced model evaluation; MLflow tracking; SHAP explanation service	22
3	Build pgvector retrieval pipeline and LangGraph analyst workflow; implement document-search, flight-query, prediction, explanation, and event-reconstruction tools	22
4	Integrate the analyst UI; create agent evaluation cases; measure groundedness, tool selection, latency, and predictive/system performance; complete demonstration and write-up	18

References

adsb.lol. (2026). *adsb.lol ADS-B API*. https://api.adsb.lol/docs.

airplanes.live. (2026). *airplanes.live ADS-B API*. https://airplanes.live/api-guide/.

Airspace Intelligence. ({n.d.}). *PRESCIENCE platform*. https://www.airspace-intelligence.com/platform

AlBassam, S. A. A., & AlShahrani, D. N. (2025). Flight delay prediction: Evaluating machine learning algorithms for enhanced accuracy. *PLOS ONE*, *20*(12), e0335141.

Amadeus. ({n.d.}). *Flight operations control*. https://amadeus.com/en/airlines/products/flight-operations-control

Assaia International AG. (n.d.). *ApronAI: Real-time visibility and predictive alerting for turnaround operations*. https://www.assaia.com/solutions/apron-ai

Beatty, R., Hsu, R., Berry, L., & Rome, J. (1999). Preliminary evaluation of flight delay propagation through an airline schedule. *Air Traffic Control Quarterly*, *7*(4), 259–270.

Belcastro, L., Marozzo, F., Talia, D., & Trunfio, P. (2016). Using scalable data mining for predicting flight delays. *ACM Transactions on Intelligent Systems and Technology*, *8*(1), 1–20.

Bureau of Transportation Statistics. (2026). *Reporting carrier on-time performance*. https://www.transtats.bts.gov/Tables.asp?DB_ID=120.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Federal Aviation Administration. (2020). *TFDM capabilities*. U.S. Department of Trans-

portation. https://www.faa.gov/air_traffic/technology/tfdm/capabilities

Federal Aviation Administration. (2024a). *Roadmap for artificial intelligence safety assurance*. Federal Aviation Administration. https://www.faa.gov/media/82891

Federal Aviation Administration. (2024b). *System wide information management (SWIM)*. https://www.faa.gov/air_traffic/technology/swim.

Federal Aviation Administration. (2024c). *NextGen*. U.S. Department of Transportation. https://www.faa.gov/newsroom/nextgen

FlightAware. ({n.d.}). *FlightAware foresight predictive flight tracking*. https://uk.flightaware.com/commercial/foresight/

Kafle, N., & Zou, B. (2016). Modeling flight delay propagation: A new analytical-econometric approach. *Transportation Research Part B: Methodological*, *93*, 520–542.

Lauderdale, T. A., Windhorst, R. D., Coppenbarger, R. A., Thipphavong, D. P., & Erzberger, H. (2024). Overview of the national airspace system (NAS) digital twin simulation environment. *AIAA AVIATION Forum and ASCEND 2024*. https://doi.org/10.2514/6.2024-4011

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems 33*, 9459–9474.

Lufthansa Systems. (2025). *NetLine/Ops++ aiOCC*. https://cdn.lhsystems.com/2025-02/2025_02_Product_information_NetLine_Ops_%2B%2B_aiOCC.pdf

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems 30*, 4765–4774.

National Aeronautics and Space Administration. (2021). *Airspace technology demonstration 2 (ATD-2) fuser*. Computer software; GitHub. https://github.com/nasa/atd2-fuser

National Centers for Environmental Information. (2026). *Integrated surface database (ISD) / global hourly*. https://www.ncei.noaa.gov/products/land-based-station/integrated-surface-database.

National Oceanic and Atmospheric Administration. (2026). *Aviation weather center METAR data API*. https://aviationweather.gov/data/api/.

OpenFlights. (2026). *OpenFlights airline database*. https://openflights.org/data.html.

OurAirports. (2026). *OurAirports open data*. https://ourairports.com/data/.

Pepper, N., Keane, A., Hodgkin, A., Gould, D., Henderson, E., Lauritsen, L., Vlahos, C., De Ath, G., Everson, R., Cannon, R., Sierra Castro, A., Korna, J., Carvell, B., & Thomas, M. (2026). A probabilistic digital twin of UK en route airspace for training and evaluating AI agents for air traffic control. *AIAA SciTech Forum*. https://doi.org/10.2514/6.2026-1794

Sabre Corporation. (n.d.). *Airline disruption management software*. https://www.sabre.com/airline-mosaic/service-suite/disruption-management/

Schäfer, M., Strohmeier, M., Lenders, V., Martinovic, I., & Wilhelm, M. (2014). Bringing up OpenSky: A large-scale ADS-B sensor network for research. *Proceedings of the 13th International Symposium on Information Processing in Sensor Networks*, 83–94.

Shelke, A. J., Shelke, R. J., Kamerkar, Y. M., & Hazarani, N. P. (2026). *FlightSense: An end-to-end MLOps platform for real-time flight delay prediction via rotation-chain propagation features and agentic conversational AI*. https://doi.org/10.48550/arXiv.2605.07364

Sternberg, A., Soares, J., Carvalho, D., & Ogasawara, E. (2017). *A review on flight delay*

prediction. https://arxiv.org/abs/1703.06118

Wilson, J., Nindra, K., Austin, I., & Sokolow, V. (2026). *From prediction to digital twin: A propagation-aware machine learning framework for flight delay modeling*. https://jonathanwilsonami.github.io/OR568_ML_Project/

Wong, J.-T., & Tsai, S.-C. (2012). A survival model for flight delay propagation. *Journal of Air Transport Management*, *23*, 5–11.